

1. Multiply a and c either from the multiplication lookup table or from a recursive call to `karatsuba()`.
2. Multiply b and d either from the multiplication lookup table or from a recursive call to `karatsuba()`.
3. Multiply $a + c$ and $b + d$ either from the multiplication lookup table or from a recursive call to `karatsuba()`.
4. Calculate step 3 – step 2 – step 1.
5. Pad the step 1 and step 4 results with zeros; then add them to step 2.

The result of step 5 is the product of x and y . The specifics of how to pad the step 1 and step 4 results with zeros are explained later in this section.

Let's ask our three recursive algorithm questions about the `karatsuba()` function:

What is the base case? Multiplying single-digit numbers, which can be done with a precomputed lookup table.

What argument is passed to the recursive function call? The a , b , c , and d values derived from the x and y arguments.

How does this argument become closer to the base case? Since a , b , c , and d are each half of the digits of x and y and themselves are used for the next recursive call's x and y arguments, the recursive call's arguments become closer and closer to the single-digit numbers the base case requires.

Our Python implementation for Karatsuba multiplication is in the *karatsubaMultiplication.py* program:

```
import math

# Create a lookup table of all single-digit multiplication products:
MULT_TABLE = {} ❶
for i in range(10):
    for j in range(10):
        MULT_TABLE[(i, j)] = i * j

def padZeros(numberString, numZeros, insertSide):
    """Return a string padded with zeros on the left or right side."""
```